

QECTOR DECODER V3

Official User Manual and Extended Reference

Quantum Error Correction Decoding Platform

Version 1.0.0

Package:	qector-decoder-v3==1.0.0
Released:	2026-08-06 (15 binary wheels, PyPI Trusted Publishing + Sigstore)
Platform:	CPython 3.9–3.13 · Windows amd64 · Linux x86_64 · macOS 11+ (arm64)
License:	PolyForm Noncommercial 1.0.0 · Commercial license for business use
Website:	https://www.qector.store

Copyright 2026 Guillaume Lessard / iD01t Productions. All rights reserved.

QECTOR CERTIFIED - PROVENANCE INTACT

Table of Contents

Front matter	3
How to read this book	3
PART I: User Manual	3
1. Introduction and what is new in v1.0.0	3
2. System requirements	4
3. Installation	5
3.1 Standard install	5
3.2 Optional extras	5
3.3 Verifying what you downloaded (strong integrity)	5
3.4 Source build (licensed / OpenCL)	6
3.5 First run	6
4. Quick start (single-syndrome decode, batch, Stim DEM, CLI)	6
4.1 Single syndrome, Python	6
4.2 Batch decode (CPU + CUDA)	7
4.3 Weighted GPU decode (accuracy path)	7
4.4 Stim DEM workflow	8
4.5 Automatic decoder recommendation	8
4.6 CLI	8
5. Decoder catalogue	8
6. The Python layer	10
7. Stim / Sinter / PyMatching / qiskit-qec	13
8. API stability tiers	13
9. GPU, CuPy and OpenCL: honest capability matrix	14
10. REST / local service and security boundary	14
11. Licensing and citation	14
12. Release history summary	14
PART II: Extended Reference (package API)	15
1. Core architecture	15
2. Decoder families (package API)	15
BP-OSD options (v1.0.0)	15
3. Routing and DemModel	15
4. AutoDecoder: 7-tier fallback	16
5. Code generators	16
6. Minimal usage examples	17
7. Package MCP server (run_mcp_server)	17
8. Hyperedge decoding	18
9. Environment variables (tuning reference)	19
10. Licensing API and key resolution	19
11. What this package is not	19
PART III: CLI Reference	20
1. qector decode	20
2. qector bench	20
3. qector serve	21

4. qector-doctor: 15-check environment diagnostic	21
PART IV: How to benchmark (methodology)	22
1. Principles: honest, reproducible, system-specific.....	22
2. Throughput via qector bench (CLI)	22
3. LER via the Python ler module.....	23
4. MCP benchmark procedure (benchmark_decoder, run_ler_benchmark).....	23
5. Reporting checklist.....	25
Appendix A: Release history summary (0.5.0 to 1.0.0)	25
Appendix B: Wheel SHA-256 digest matrix (1.0.0, canonical)	26
Appendix C: License terms summary	26
Appendix D: Citation	27
Appendix E: Licensing	27

Front matter

How to read this book

This edition covers **the `qector-decoder-v3` Python package and its `qector` command-line tool only**. The companion *Qector Decoder Workbench* graphical application (Windows/Linux) has its own manual and is documented separately; it is **out of scope** here.

- **PART I** is the user manual: install, run, decode. Read this first.
- **PART II** is the package reference: every family, every module, every environment variable.
- **PART III** is the CLI reference: `qector decode`, `qector bench`, `qector serve`, `qector-doctor`.
- **PART IV** is the benchmarking guide: how to measure throughput and logical error rate on your own hardware, honestly and reproducibly (methodology only; this manual publishes no benchmark results).
- Appendices carry the wheel digests, history, license and citations.

Code blocks, tables and call-out notes are used consistently. **Honest-note** call-outs mark statements that contradict typical marketing phrasing. They matter.

Honesty and scope. Decoder behaviour and accuracy depend on the machine they run on, the exact check topology, the batch, the noise regime and the driver state. QECTOR ships tools to measure this on your own hardware; no claim is made that any single number is "ultimate".

PART I: User Manual

1. Introduction and what is new in v1.0.0

QECTOR Decoder v3 is a production-grade, source-available quantum-error-correction decoding platform: a compiled Rust core (PyO3) with a Python 3.9 to 3.13 surface. It decodes syndromes from stabiliser/qLDPC circuits with 25+ decoder configurations, feeds Stim/Sinter workflows, integrates with PyMatching and qiskit-qec, and can sit behind an MCP server or a local CLI.

v1.0.0 is the first stable release. From this version the public API is governed by the stability tiers of Part I section 8; bit-level compatibility is documented and an audit trail is kept.

What is new in 1.0.0	Detail
First stable (v1) release	Semantic-versioning frozen; tiering of the API (Part I section 8)
Ecosystem entry points	Five Sinter decoders (<code>qector_blossom</code> , <code>qector_belief</code> , <code>qector_unionfind</code> , and more) and the <code>qiskit-qec</code> plugin are registered entry points; <code>sinter.collect(decoders=[...])</code> works without <code>custom_decoders=</code>
<code>pymatching</code> submodule shim	<code>from qector_decoder_v3.pymatching import Matching</code>
New decoder families	<code>AmbiguityClusterDecoder</code> , <code>TwoStageDecoder</code> , <code>ColourCodeDecoder</code>
Relay-BP	Layered serial BP schedule for qLDPC (<code>bp_method="relay"</code>); each check sees the freshest messages
CS-OSD(λ , w) + LLR damping	Formal combination-sweep OSD in <code>bposd.py</code> with configurable <code>osd_lambda</code> and message <code>damping</code>
Weighted Union-Find on GPU	<code>CUDABatchDecoder</code> / <code>OpenCLBatchDecoder</code> accept <code>edge_weights</code> ; both kernels agree
<code>CUDABatchDecoder(precision="f64")</code>	Double-precision weighted growth kernel for accuracy-critical workloads
<code>ColourCodeDecoder(method="cluster_bposd")</code>	Weighted UF growth over the undecomposed hypergraph DEM with spanning-tree peeling, then BP-OSD on residuals
<code>DemModel.make_decoder</code>	Covers all nine shipped native families from the DEM, with weights carried through
<code>qector</code> CLI + <code>qector-doctor</code>	<code>qector decode/serve</code> and a 15-check environment diagnostic
<code>SparseBlossomDecoder</code> zero-allocation	Hot path is now zero-allocation via thread-local <code>SbScratch</code>
Licence hardening	v2 tokens carry tier + expiry; malformed tokens return <code>False</code> ; unreadable key files report <i>invalid</i>
Rust core crash safety	Six panic-to-abort paths removed (gRPC/CUDA mutex-poison propagation, swallowed CUDA async errors, <code>Bernoulli::new</code> <code>unwrap</code> , cascade-decoder <code>expect</code>)
<code>generate_parity_check_matrix()</code>	Now bound at module level for custom code construction

2. System requirements

Requirement	Supported	Notes
Python	3.9, 3.10, 3.11, 3.12, 3.13 (CPython)	<code>requires-python >=3.9</code>
OS / wheels	Windows (win_amd64), Linux (manylinux_2_17 x86_64), macOS 11.0+ (arm64)	15 binary wheels; no sdist
NumPy	<code>>=1.24.0, <2.3</code>	runtime dependency
Optional	CuPy (<code>pip install qector-decoder-v3[cuda,cupy]</code>) for batched GPU-resident BP; CUDA toolkit only for source builds	CUDA path ships in the wheel; OpenCL kernels must be built from source
RAM	8 GiB recommended; batch sweeps scale with batch size	<code>decode_mmap</code> exists for out-of-core
Display	none	headless-safe in CLI and server modes

GPU note. The standard wheel ships a working CUDA path (`CUDABatchDecoder`). On Windows the NVIDIA driver is used in WDDM mode; compute servers using TCC typically see higher performance. OpenCL kernels are **not compiled** into any wheel. `openc1_is_available()` returns `False` everywhere, even where an OpenCL device exists. Use the source build if you need OpenCL.

3. Installation

3.1 Standard install

```
python -m pip install "qector-decoder-v3==1.0.0"
```

All in-wheel decoders (Union-Find, Blossom, BP-OSD, BeliefMatching, and more) run on the CPU core; no optional dependency is required.

3.2 Optional extras

The following optional extras are available for installation:

Extra	What it installs	Needed for
<code>cuda</code>	CUDA runtime bindings	NVIDIA GPU batch decode via <code>CUDABatchDecoder</code>
<code>cupy</code>	CuPy	GPU-resident batched BP (<code>bp_cupy</code>)
<code>cupy-cuda12x</code>	CuPy CUDA 12-specific build	CUDA 12-optimized GPU batch decoding
<code>stim</code>	Stim, Sinter, PyMatching, LDPC ecosystem	Stim workflows and entry points
<code>openc1</code>	OpenCL bindings (wheel forbids kernels)	building from source with the OpenCL feature
<code>stripe</code>	(payment) licence validation tooling	commercial tiers
<code>bench</code>	Benchmark utilities	honest benchmarking tools
<code>pypy</code>	PyPy-specific optimizations	PyPy runtime optimizations
<code>dev</code>	Development tools (linting, testing)	contributors and package maintainers
<code>all</code>	all of the above guide-only superset	convenience

To install all optional features:

```
python -m pip install "qector-decoder-v3[all]==1.0.0"
```

3.3 Verifying what you downloaded (strong integrity)

Every wheel is published with **Trusted Publishing + Sigstore attestations**. Two checks are recommended:

SHA-256 (PyPI shows the digest):

bash

```
# at the PyPI page, the file digest is listed; verify on disk:
$ python -c "import hashlib; print(hashlib.sha256(open('qector_decoder_v3-1.0.0-cp311-cp311-win_amd64.whl', 'rb').read()).hexdigest())"
7f198c7d9ca8f28c461f907d2ec4f41464198446abc2239c894b550bce7d4f98 # cp311 win_amd64
```

Sigstore (needs `cosign`): verify the signature against the release tag:

bash

```
cosign verify-blob --certificate ... --signature ... qector_decoder_v3-1.0.0-cp311-cp311-win_amd64.whl
```

All 15 wheels SHA-256 digests are listed in Appendix B.

3.4 Source build (licensed / OpenCL)

Building from source requires the Rust toolchain (see the README). The `openc1` cargo feature compiles the OpenCL batch kernels; the CPU and CUDA paths in the wheel are the same decoding arithmetic as a source build.

3.5 First run

bash

```
qector-doctor # 15-check environment diagnostic; reports WHY a decoder is unavailable
qector decode --help
python -c "import qector_decoder_v3; print(qector_decoder_v3.__version__)"
```

The first import prints a licensing notice; set `QECTOR_SILENT=1` to suppress it.

4. Quick start (single-syndrome decode, batch, Stim DEM, CLI)

4.1 Single syndrome, Python

python

```
from qector_decoder_v3 import UnionFindDecoder, BlossomDecoder

checks = [(0, 1), (0, 2), (1, 3), (2, 3)] # parity-check hyperedges
n_qubits = 4
syn = [1, 0, 1, 0] # must satisfy len(syn) == len(checks)

fast = UnionFindDecoder(checks, n_qubits)
mwpm = BlossomDecoder(checks, n_qubits)

print(fast.decode(syn)) # GF(2)-faithful correction (H @ corr == syndrome, mod 2)
print(mwpm.decode(syn))
```

4.2 Batch decode (CPU + CUDA)

```
python

from qector_decoder_v3 import BatchDecoder, CUDABatchDecoder

checks = [(0, 1), (0, 2), (1, 2), (2, 3), (3, 4)]
n_qubits = 5
cpu = BatchDecoder(checks, n_qubits)      # AVX2 path
gpu = None
if CUDABatchDecoder.is_available():      # True on a CUDA device
    gpu = CUDABatchDecoder(checks, n_qubits)

syndromes = [[1,0,1,0,0], [0,1,0,1,0]]
print(cpu.parallel_batch_decode(syndromes))
if gpu is not None:
    print(gpu.batch_decode(syndromes))    # matches CPU, syndrome-faithful
```

Note. Construct `CUDABatchDecoder` with both `check_to_qubits` **and** `n_qubits` ;

a missing argument raises rather than silently mis-firing.

4.3 Weighted GPU decode (accuracy path)

```
python

from qector_decoder_v3 import dem, CUDABatchDecoder

model = dem.from_stim(circuit.detector_error_model(decompose_errors=True))
graph = model.collapse_to_graph()

gpu = CUDABatchDecoder(
    graph.check_to_qubits(),
    graph.num_errors,
    graph.weights().tolist(),    # log((1-p)/p) per mechanism
)
```

Honest note: Without `edge_weights` the GPU kernels run unweighted cluster growth, which cannot distinguish a $p = 10^{-4}$ mechanism from a $p = 10^{-2}$ one; on circuit-level noise that costs several times the logical error rate. The `precision="f64"` option gives double-precision weighted growth for accuracy-critical workloads.

4.4 Stim DEM workflow

```
python

import stim
from qector_decoder_v3 import BlossomDecoder, DemModel

circuit = stim.Circuit.generated(
    "repetition_code:memory", distance=5, rounds=3,
    before_round_data_depolarization=0.01
)
dem = circuit.detector_error_model(add_detectors_for_gate_errors=True)
model = DemModel.from_stim(dem)
decoder = model.make_decoder("blossom")      # or "union_find", "bposd", ...
sampler = circuit.compile_detector_sampler()
syndromes, observables = sampler.sample(shots=1000, separate_observables=True)
for s in syndromes:
    corr = decoder.decode(s)
```

4.5 Automatic decoder recommendation

```
python

from qector_decoder_v3 import recommend_decoder
print(recommend_decoder(code_family="surface", distance=5, priority="accuracy"))
# BlossomDecoder
```

Workload-sensitive: regeneration per code/hardware/batch, see Part II section 3.

4.6 CLI

```
bash

qector decode --dem model.dem --syndrome "0,1,0,1"
qector serve --port 8080                      # local REST boundary
qector-doctor                                # 15-check diagnostic
```

Full CLI reference in Part III.

5. Decoder catalogue

Faithfulness invariant. Every decoder in every tier returns $GF(2)$ -faithful corrections: the accepted correction c always satisfies $H @ c == syndrome \pmod{2}$. Families that violate this for a code raise a clear error, never a silent wrong result.

Decoder / entry	Best use	Tier		
<code>UnionFindDecoder</code> (aliases <code>UF</code> , <code>union_find</code>)	Low-latency approximate decode	Stable		
<code>FastUnionFindDecoder</code>	Optimized Union-Find hot path	Stable		
<code>BlossomDecoder</code>	Exact MWPM; runs PyMatching-parity validation	Stable		
<code>SparseBlossomDecoder</code>	Faster near-optimal matching (zero-allocation hot path in v1.0.0)	Experimental		
<code>BatchDecoder</code> / <code>CPUBatchDecoder</code>	CPU batch Monte-Carlo sweeps (AVX2)	Stable		
<code>CUDABatchDecoder</code>	Optional CUDA batch decode, <code>edge_weights</code> , <code>precision="f64"</code> , <code>output == CPU batch result</code>	Workload-sensitive (runtime dependent)		
<code>CUDABpOsdDecoder</code>	CUDA BP-OSD batch decode	Build/runtime dependent		
<code>OpenCLBatchDecoder</code>	OpenCL batch decode (<code>edge_weights</code>)	Build dependent (not in wheels)		
<code>SpaceTimeDecoder</code>	3D space-time (multi-round) decoding	Experimental		
<code>BpOsdDecoder</code> / <code>BPOSDDecoder</code>	LDPC / qLDPC decoding, Relay-BP schedules, CS-OSD, LLR damping	Experimental		
<code>BeliefMatching</code>	Correlated-noise accuracy mode	Workload-sensitive		
<code>LookupTableDecoder</code>	Tiny codes, O(1) table lookup	Stable		
<code>PredecodedDecoder</code>	Easy-syndrome prefiltering	Experimental		
<code>AutoDecoder</code> / <code>AutoRouterPolicy</code>	Routes to a concrete decoder by analysis	Workload-sensitive		
<code>HybridDecoder</code>	GNN pre-decoder + sparse blossom	Experimental		
<code>DecoderPool</code> / <code>get_decoder</code> / <code>decode_mmap</code>	Scale-out batch, cached factory, out-of-core	Stable		
<code>Workbench</code>	High-level multi-decoder orchestration (Python)	Stable		
<code>SlidingWindowDecoder</code> / <code>StreamingDecoder</code>	Multi-round streaming workflows	Experimental		
<code>NeuralPredecoder</code> / <code>GNNPredecoder</code> / <code>GNNTrainer</code>	Learned predecoder front-ends	Research		
<code>AmbiguityClusterDecoder</code>	BP +	LLR	partition + exact per-cluster enumeration	Experimental
<code>TwoStageDecoder</code>	Decoupled X and Z sector matching	Experimental		
<code>ColourCodeDecoder</code>	BP-OSD over the <i>undecomposed</i> colour-code hypergraph; opt-in <code>method="cluster_bposd"</code>	Experimental		

Decoder / entry	Best use	Tier
LERBenchmark	Logical error rate evaluation with Wilson CI	Experimental

Honest notes

- Union-Find is a fast approximate path (higher LER than exact MWPM), best as triage or a performance lever.
- BlossomDecoder is weight-optimal exact MWPM; it reaches PyMatching accuracy but is **not faster**.
- SparseBlossom is region-growing *near-optimal*, not exact. In v1.0.0 its hot path is zero-allocation via thread-local SbScratch.
- BpOsdDecoder is a self-contained BP-OSD for arbitrary GF(2) check matrices, always syndrome-faithful. v1.0.0 adds Relay-BP (`bp_method="relay"`), CS-OSD with configurable `osd_lambda`, and LLR message `damping`.
- CUDABatchDecoder output is bit-identical to the CPU batch result and syndrome-faithful. The `precision="f64"` option gives double-precision weighted growth.
- OpenCLBatchDecoder reports unavailable in the standard wheels (no kernels compiled); enable it with a source build.
- ColourCodeDecoder default method is `bposd` (plain BP-OSD). The opt-in `method="cluster_bposd"` uses weighted UF growth with spanning-tree peeling, then BP-OSD on residuals.

6. The Python layer

All additive modules are NumPy+stdlib-first and degrade cleanly when CuPy is absent. `import qector_decoder_v3` can never break on a CPU-only box.

The Python layer is **additive, not required**. Every public symbol at the top level (decoders, routing, `DemModel`, license API, `ler`, `run_mcp_server`, `__version__`, the `dem` submodule alias) is re-exported from `qector_decoder_v3/__init__.py` and works without any of the optional extras being installed. Optional features are gated behind importable submodules so `import qector_decoder_v3.bposd` or `import qector_decoder_v3.predecoder` is the only place a missing dependency can surface, and a missing `cupy` or `stim` import is reported cleanly through `hasattr` probes, not as an `ImportError` at top-level import time. The CPU decode path and the stable table below are always available on every wheel.

Top-level surface (importable directly from `qector_decoder_v3`):

Symbol	Notes
<code>UnionFindDecoder</code> , <code>BlossomDecoder</code> , <code>SparseBlossomDecoder</code>	low-latency, exact-MWPM, near-optimal sparse hot path
<code>BatchDecoder</code> , <code>CUDABatchDecoder</code>	CPU AVX2 batch and optional CUDA batch
<code>DemModel</code> , <code>recommend_decoder</code>	DEM-driven construction and routing (Section 4.4)
<code>ColourCodeDecoder</code>	colour-code hypergraph decoder
<code>set_license_key</code> , <code>set_license_key_file</code> , <code>get_license_info</code> , <code>verify_license_token</code>	license API (Part II section 10)
<code>ler</code>	LER measurement module (Part IV section 3)
<code>dem</code>	submodule alias for <code>DemModel</code> and DEM helpers
<code>run_mcp_server</code>	MCP server entry point (Part II section 7)
<code>__version__</code>	dunder; string form of the package version

Pattern. Most code only needs the top-level surface. Reach into a submodule (`qector_decoder_v3.bposd` , `.codes` , `.rest_api` , ...) when you want a feature that is not stable, not always available, or that is intentionally out of the default import path.

Module	Purpose	Tier
<code>gpu_backend</code>	One policy-aware GPU capability foundation (<code>has_cupy</code> , <code>has_cuda_rust</code> , <code>gpu_available</code> , <code>get_backend</code> , <code>get_array_module</code>)	Stable
<code>bp_cupy</code>	GPU-resident batched belief propagation (<code>BatchedBpDecoder</code> , <code>batched_bp_decode</code>); the NumPy path is bit-identical for a single shot	Workload-sensitive
<code>dem</code>	Detector-error-model surface (<code>DemModel.from_stim</code> , <code>collapse_to_graph</code> , <code>make_decoder</code>) carrying edge weights into every decoder	Stable
<code>routing</code>	Decoder selection (<code>recommend_decoder</code> , <code>AutoRouter</code>) inspecting the actual check structure; routes hyperedge/qLDPC to BP-OSD	Stable
<code>streaming</code>	Dependency-free multi-round orchestration (<code>StreamingSession</code> , <code>sliding_window_decode</code>) in the phenomenological perfect-measurement regime	Experimental
<code>autodebug</code>	Self-debugging layer behind <code>AutoDecoder</code> (<code>probe_decoders</code> , <code>resilient_single_decode</code> , <code>run_self_diagnostics</code>)	Stable
<code>scale-out</code>	<code>DecoderPool</code> (multi-process, auto-Rayon fallback on Windows), <code>get_decoder</code> cached factory, <code>decode_mmap</code> out-of-core, <code>Workbench</code> orchestrator	Stable
<code>bposd</code>	BP-OSD with Relay-BP, CS-OSD(λ , w), LLR damping	Workload-sensitive
<code>belief_matching</code>	<code>BeliefMatching.from_numpy_h</code> faithful length-n_qubits output	Workload-sensitive
<code>predecoder</code>	<code>PredecodedDecoder</code> , <code>NeuralPredecoder</code> , <code>GNNPredecoder</code> , <code>GNNTrainer</code>	Experimental (GNN/Neural: Research)
<code>codes</code>	Code generators: repetition, ring, surface, toric, heavy_hex, bicycle, bivariate_bicycle, color_code, hypergraph_product	Stable
<code>stim_compat</code>	<code>from_stim_detector_error_model</code> , <code>stim_decoder_from_dem</code>	Workload-sensitive
<code>sinter_compat</code>	<code>qector_sinter_decoders()</code> with entry points registered	Workload-sensitive
<code>pymatching</code> / <code>pymatching_compat</code>	<code>Matching</code> drop-in shim (submodule and attribute spelling)	Workload-sensitive
<code>qiskit_plugin</code>	qiskit-qec entry point (registered in v1.0.0)	Workload-sensitive
<code>rest_api</code>	Local FastAPI decoding service (127.0.0.1 only)	Experimental
<code>license</code>	<code>verify_license_token</code> , <code>set_license_key</code> , <code>set_license_key_file</code> , <code>get_license_info</code>	Stable
<code>ler</code>	LER measurement (<code>estimate_ler</code> , <code>estimate_ler_circuit_level</code> , <code>assert_comparable</code> , <code>wilson_ci</code> , <code>run_threshold_sweep</code> , <code>run_memory_experiment</code> , <code>run_competitive_suite</code>); see Part IV section 3	Stable

Usage pattern (top-level only, CPU, no optional deps):

```
python

from qector_decoder_v3 import (
    UnionFindDecoder, BlossomDecoder, # decoders
    DemModel, recommend_decoder,      # DEM + routing
    run_mcp_server, get_license_info,  # service surface
)
import qector_decoder_v3 as qv        # __version__ lives here
print(qv.__version__)
```

Tier definitions are in section 8 below; this table maps each module to its tier so the developer can decide whether the API is something to depend on.

7. Stim / Sinter / PyMatching / qiskit-qec

Hook	API	Reference toolchain
Stim	<code>stim_compat : from_stim_detector_error_model , stim_decoder_from_dem</code>	Stim v1.16.0
Sinter	<code>sinter_compat : qector_sinter_decoders()</code> provides <code>qector_blossom</code> , <code>qector_belief</code> , <code>qect-or_unionfind</code> , and more; entry points registered , so <code>sinter.collect(decoders=["qector_blossom", ...])</code> needs no <code>custom_decoders=</code>	Sinter
PyMatch-ing shim	<code>from qector_decoder_v3.pymatching_compat import Matching</code> and the submodule spelling <code>from qector_decoder_v3.pymatching import Matching</code> ; backed by the QECTOR blossom decoder	PyMatching v2.4.0
qiskit-qec	the <code>qiskit_plugin</code> entry point (registered in v1.0.0)	Qiskit

```
python

import sinter
results = sinter.collect(
    decoders=["qector_blossom", "qector_unionfind"],
    circuit=circuit, shots=1000
)
```

8. API stability tiers

Tier	Meaning	Members (representative)
Stable	Usable across 1.x; bug-fix changes only	Core decoders, code generators, <code>DecoderPool</code> , <code>get_decoder</code> , <code>decode_mmap</code> , <code>Workbench</code> , <code>dem.make_decoder</code> , source build
Workload-sensitive	Valid, but claims must be regenerated per code/hardware/ batch	<code>BeliefMatching</code> , <code>BP-OSD</code> , <code>AutoDecoder/ AutoRouter</code> , <code>CUDA batch</code> , <code>Stim/Sinter/ PyMatching</code> helpers
Experi- mental / preview	REST/gRPC/MCP services, metrics exporter, hybrid, two- stage, ambiguity-cluster, colour-code, GNN/neural pre- decoders, hosted API, OEM embedding, OpenCL batch	Deployment or build review required
Internal de- tail	Native core internals and fallback heuristics; may change without notice	Rust-core heuristics

9. GPU, CuPy and OpenCL: honest capability matrix

Compute path	In standard wheel?	Probe	Notes
CPU decoders	Always	N/A	Always available
CUDA <code>CUDABatchDecoder</code>	Yes (CUDA core)	<code>has_cuda_rust()</code> , <code>cuda_is_available()</code>	Available on a CUDA device; output matches the CPU batch result; optional <code>edge_weights</code> ; optional <code>precision="f64"</code>
CUDA-BP <code>CUDAB-p0sdDecoder</code>	Build-depend-ent	<code>cuda_is_available()</code>	CUDA builds only; single-shot convenience via <code>.decode()</code>
CuPy <code>bp_cupy</code>	Optional (<code>pip install cupy</code>)	<code>has_cupy()</code> , <code>gpu_avail-able()</code>	Device present + CuPy
OpenCL <code>OpenCL-BatchDecoder</code>	No (no kernels)	<code>opencl_is_available()</code>	<code>False</code> even with an OpenCL device; source build re-quired

You lose nothing on the standard wheel: the CPU and CUDA paths do the same decoding, and the CUDA path is correctness-verified against the CPU batch result.

10. REST / local service and security boundary

Together with the MCP server (Part II section 7), a local REST service is provided by `qector serve` (Part III section 3). All such surfaces share the same security boundary:

- Bind to `127.0.0.1` only, by default, and never expose it directly to the public internet (no TLS termination, no coarse input throttling, no authN/authZ layer).
- Validate every request: syndrome lengths, decoder families, content length (10 MB cap). Malformed JSON-RPC/REST calls return an error object, never a crash.
- Treat the local service as **experimental / preview** tier: for local, single-user, controlled use and prototyping. For SaaS/hosted deployment you must consult the licensing terms first (commercial licence required, see section 11) and add your own termination, rate limiting, and authentication around it.

11. Licensing and citation

The software is **source-available under the PolyForm Noncommercial License 1.0.0**: free for personal, academic, educational and non-commercial research; **commercial use, funded institutional work, SaaS/hosted API deployment, OEM integration or paid consulting requires a commercial licence** (<https://qector.store/pricing>, admin@qector.store).

Please cite using Appendix D.

12. Release history summary

Detailed changelog in Appendix A (0.5.0 to 1.0.0). Highlights beyond 0.7.0 are listed in section 1 of this Part.

,

PART II: Extended Reference (package API)

1. Core architecture

Rust extension (PyO3), Python 3.9 to 3.13 surface. The Python layer is a thin but essential tier: layout, routing, scale-out, batch plumbing and the `dem` weight carrier. The Rust core owns decoding (Blossom, Union-Find, BP, OSD, cascade), the licence kernel (Ed25519) and the batch kernels (CPU AVX2, CUDA).

```
qector_decoder_v3/
+-- Rust core (proprietary, injected during CI build or under license)
|   +-- Union-Find / Blossom / SparseBlossom engines
|   +-- CPU batch engine (SIMD-accelerated on x86)
|   +-- CUDA / OpenCL batch paths
|   +-- DEM collapse and Stim integration
|
+-- Python layer (open source in this repository)
    +-- __init__.py, backend.py, dem.py
    +-- belief_matching.py, bposd.py
    +-- predecoder.py, codes.py
    +-- stim_compat.py, sinter_compat.py
    +-- qiskit_plugin.py, rest_api.py
    +-- workbench.py, license.py, ler.py
```

2. Decoder families (package API)

Families are available under short names (`blossom`, `blossom_bp`, ...) and long class names (`BlossomDecoder`, ...) as listed in the catalogue in Part I section 5. Constructors follow the pattern:

```
python

BlossomDecoder(checks_or_H, n_qubits=None, ...)
UnionFindDecoder(checks_or_H, n_qubits=None, ...)
BpOsdDecoder(H, error_rate=0.05, osd_order=0, bp_method="exact",
             damping=0.0, osd_lambda=24, ...) # arbitrary GF(2) matrices
```

For circuit DEM-driven construction the preferred path is `DemModel.make_decoder(family)` (section 3).

BP-OSD options (v1.0.0)

Option	Default	Effect
<code>bp_method</code>	"exact"	"exact" (log-domain sum-product), "min_sum", or "relay" (layered serial)
<code>osd_order</code>	0	0, 1, or 2 for combination-sweep OSD
<code>osd_lambda</code>	24	Number of free bits closest to the reliability cut-off for CS-OSD
<code>damping</code>	0.0	LLR message damping: $m = (1-d)*m_{\text{new}} + d*m_{\text{old}}$ on <code>min_sum / sum_product</code>

3. Routing and `DemModel`

- `from qector_decoder_v3.dem import DemModel` with `from_stim` / `from_parities` / `make_decoder`, carrying edge weights through to every decoder; `make_decoder` covers all nine shipped native families.
- `DemModel.DECODER_KINDS` enumerates all constructible families.

- `recommend_decoder(code_family, distance, priority)` inspects the actual check structure; hyperedge/qLD-PC is routed to BP-OSD, surface-type to Blossom/UF.
- `generate_parity_check_matrix()` is bound at module level for custom code construction.

4. AutoDecoder: 7-tier fallback

Behind `AutoDecoder / AutoRouter`, `autodebug` probes candidate decoder families against your check structure and, on each decode error, walks the 7 tiers in a deterministic order (probe, single, cascade, legacy, ...), always landing on a family that can return a faithful correction or a clear error. `run_self_diagnostics()` gives a full report.

Tier	Backend	Description
1	CUDA Batch	GPU batch decoding via NVRTC-compiled kernels
2	OpenCL Batch	Cross-vendor GPU batch decoding
3	CPU Rayon	Multi-threaded parallel CPU batch decoding
4	CPU Batch	Single-threaded CPU batch decoding
5	CPU Single	Per-syndrome CPU decoding
6	Blossom	Exact MWPM fallback (guaranteed correctness)
7	Lookup Table / Python	Pure-Python last-resort fallback

Key features:

- **Automatic error trapping:** Hardware exceptions (CUDA OOM, driver crashes, memory limits) are caught, logged, and bypassed transparently.
- **Health scoring:** Each backend tracks its health status. Failed backends are automatically suspended.
- **Seamless recovery:** `reset_backend_health()` re-enables all backends for dynamic recovery.
- **Diagnostic logging:** All fallback events and error details are recorded for debugging.

5. Code generators

The code-generator module builds repetition, surface, ring, colour-code, bicycle, bivariate-bicycle, hypergraph-product, heavy-hex, and toric lattices for quick experiments. Exact listing is in the API docs of the installed wheel (`help(qector_decoder_v3)` beats a printed book); the generators are Stable tier.

6. Minimal usage examples

```
python

# Example: DEM-driven construction with routing
from qector_decoder_v3 import DemModel, recommend_decoder

dem = DemModel.from_parities(checks, n_qubits, edge_weights={...}) # weights carried
decoder = dem.make_decoder("blossom") # or "union_find", "bposd", ...
print(recommend_decoder(code_family="surface", distance=5, priority="accuracy"))

# Example: BP-OSD with Relay-BP and CS-OSD
from qector_decoder_v3.bposd import BpOsdDecoder
decoder = BpOsdDecoder(H, error_rate=0.05, bp_method="relay",
                      osd_order=1, osd_lambda=24, damping=0.1)

# Example: Colour-code with cluster_bposd
from qector_decoder_v3 import ColourCodeDecoder
decoder = ColourCodeDecoder(checks, method="cluster_bposd")
```

Use the code blocks of Part I section 4 verbatim as your templates: single, batch, DEM, routing.

7. Package MCP server (`run_mcp_server`)

Every wheel ships the MCP server (JSON-RPC 2.0 over stdio), no extra feature flag, no install:

```
python

from qector_decoder_v3 import run_mcp_server
run_mcp_server()
```

```
bash

python -c "import qector_decoder_v3; qector_decoder_v3.run_mcp_server()"
```

A ready-made client configuration lives in `mcp.json`. The server advertises 13 tools, all verified against the released wheel:

Tool	Purpose
<code>decode_syndrome</code>	Decode a syndrome with any decoder family (Union-Find, Blossom, SparseBlossom, BP-OSD, Cascade, Hybrid, ...)
<code>batch_decode</code>	Batch-decode multiple syndromes in parallel
<code>decode_hyperedge</code>	Hyperedge / qLDPC decoding (bypasses the graph-only UF restriction)
<code>decode_syndrome_blossom</code>	Exact Blossom (MWPM) single decode
<code>batch_decode_blossom</code>	Exact Blossom (MWPM) batch decode
<code>decode_syndrome_cascade</code>	Hybrid cascading decoder (UF pre-filter escalating to Blossom)
<code>benchmark_decoder</code>	Run a performance benchmark for a decoder family
<code>run_ler_benchmark</code>	Run the LER (logical error rate) benchmark across code distances
<code>get_decoder_info</code>	Decoder configuration, version info, family listing
<code>get_backend_health</code>	Backend health across the 7 fallback tiers
<code>clear_decoder_cache</code>	Clear the decoder factory cache
<code>get_server_env</code>	Effective QECTOR environment variables
<code>recommend_decoder</code>	Decoder recommendation by code topology and priority

The stdio reader enforces a 10 MB content limit and validates syndrome lengths and decoder types, returning JSON-RPC errors instead of crashing. Scope: local/controlled use; **not** hardened for public SaaS exposure (see Part I section 10).

8. Hyperedge decoding

Graphlike Union-Find rejects high-degree incidence matrices; use the explicit hyperedge path:

```
python

import numpy as np
from qector_decoder_v3 import BlossomDecoder, SparseBlossomDecoder, BpOsdDecoder

def decode_hyperedge(checks, n_qubits, syndrome, kind="Blossom", **opts):
    syn = np.asarray(syndrome, dtype=np.uint8).ravel()
    if kind.lower() in ("blossom",):
        return BlossomDecoder(checks, n_qubits).decode(syn)
    if kind.lower() in ("sparse_blossom", "sparseblossom"):
        return SparseBlossomDecoder(checks, n_qubits).decode(syn)
    if kind.lower() in ("bposd", "bp_osd"):
        H = np.zeros((len(checks), n_qubits), dtype=np.uint8)
        for i, c in enumerate(checks):
            for q in c:
                if 0 <= q < n_qubits:
                    H[i, q] ^= 1
        return BpOsdDecoder(H, error_rate=opts.get("error_rate", 0.05),
                           osd_order=opts.get("osd_order", 0)).decode(syn)
    raise ValueError(f"unsupported kind: {kind}")
```

MCP + fallback pattern: try the package MCP `decode_hyperedge` tool, or `BlossomDecoder` / `BpOsdDecoder` directly (in-wheel, always available).

9. Environment variables (tuning reference)

Variable	Default	Effect
<code>QECTOR_LICENSE</code>	(unset)	Ed25519 token (overrides key file)
<code>QECTOR_LICENSE_KEY</code>	(unset)	inline licence key; resolved before the file-based keys
<code>QECTOR_LICENSE_FILE</code>	<code>~/.qector/li- cense.key</code>	path to a licence key file; an unreadable file is <i>invalid</i> , never a silent downgrade
<code>QECTOR_SILENT</code>	(unset)	<code>1</code> suppresses the startup licensing notice
<code>QECTOR_ENFORCE</code>	(unset)	<code>1</code> enables hard license gating (rejects decode on invalid/expired)
<code>QECTOR_CUDA_DEVICE_ID</code>	<code>0</code>	CUDA device index for the native kernels
<code>QECTOR_OPENCL_DEVICE_ALLOW</code>	(unset)	comma-separated device-name substrings (source builds/OpenCL only)
<code>QECTOR_BLOSSOM_K_MULT</code>	<code>2.0</code>	candidate-neighbour multiplier (changes matching quality)
<code>QECTOR_BLOSSOM_INTRA_PAR</code>	<code>auto</code>	intra-decode parallelism for blossom (performance)
<code>QECTOR_BLOSSOM_INTRA_THREADS</code>	(unset)	dedicated Rayon pool size for the blossom hot path
<code>QECTOR_DATA_DIR</code>	per-user	relocate <code>logs</code> , <code>exports</code> , <code>settings</code> of the CLI/service
<code>QECTOR_DISABLE_OPENCL</code>	(unset)	<code>1</code> skips OpenCL probing

Only `QECTOR_BLOSSOM_K_MULT` (and `QECTOR_OPENCL_DEVICE_ALLOW`, where kernels exist) change matching quality; the rest are performance or hygiene knobs.

10. Licensing API and key resolution

```
python
from qector_decoder_v3 import set_license_key, set_license_key_file, get_license_info
set_license_key(token)           # programmatic; raises ValueError if rejected
set_license_key_file("/path/to/key") # or file
info = get_license_info()
print(info["tier"], info["key_status"]) # e.g. ('enterprise', 'valid')
```

The core resolves a key on its own in this order: `QECTOR_LICENSE_KEY` then `QECTOR_LICENSE_FILE` then `~/.qector/license.key`. Prefer a file in deployments. The key then never lands in a process listing or shell history. Check `key_status == "valid"` **and** the tier, never just the tier: a set-but-unreadable `QECTOR_LICENSE_FILE` is reported as *invalid*, not as a silent Community fallback. Offline verification: `verify_license_token` (Ed25519). Malformed tokens return `False`; they never raise.

v2 tokens carry `tier` and `exp` (expiry) inside the Ed25519 signature. Use `license_claims()` to access verified, unexpired claims.

11. What this package is not

- **Not** a circuit simulation engine (bring your samples: Stim, sinter, your own).
- **Not** a GUI. The Workbench app is covered by its own manual.

- **Not** a SaaS-ready REST/gRPC server. The rest/gRPC/MCP surfaces are designed for local controlled use and experimental deployments only.
- **Not** a security product for untrusted inputs at Internet scale.

PART III: CLI Reference

1. qector decode

Decode one syndrome or a DEM file.

```

qector decode --dem model.dem --syndrome "0,1,0,1"
qector decode --checks "0,1:0,2:1,2" --qubits 4 --syndrome "1,0,1,0" --decoder blossom

```

Flag	Meaning
--dem <file>	detector-error-model file (Stim DEM JSON or text)
--checks , --qubits	inline check hyperedges + qubit count
--syndrome	comma-separated 0/1 syndrome; length must equal the check count
--decoder	family (union_find , fast_union_find , blossom , sparseblossom , bposd , cascade , auto); default auto
--json	machine-readable output (correction, weight, backend)
--seed	determinism seed

Exit code 0 = faithful correction produced, 2 = license/tier gating, 3 = malformed input, 4 = decoder unavailable (see qector-doctor for why).

2. qector bench

Quick throughput measurement on a generated rotated surface-code circuit. The circuit is built in-memory from Stim with the requested distance, rounds and physical noise, then shots are decoded with the chosen family. The command prints a single honest rate line (e.g. shots/s for the given machine state); no claim is attached to it.

```

qector bench --distance 5 --rounds 5 --shots 10000 --decoder blossom --noise 0.001
qector bench -d 7 -r 5 -s 10000 -d blossom -n 0.001

```

Flag	Default	Meaning
<code>--distance / -d</code>	5	surface-code distance
<code>--rounds / -r</code>	5	syndrome-extraction rounds
<code>--shots / -s</code>	10,000	number of shots to decode
<code>--decoder</code>	blossom	decoder family (see <code>qector decode --help</code>)
<code>--noise / -n</code>	0.001	per-gate physical error probability used by the generated circuit

Use the same `--noise` for any two runs you intend to compare. Results are machine-, driver- and power-state-conditional; see Part IV section 1.

3. qector serve

Local REST service (127.0.0.1 only by default).

```
bash
qector serve --port 8080
```

Endpoints: `POST /decode`, `POST /batch`, `GET /health`, `GET /env`, `GET|POST /recommend`. TLS/CORS header checks are **not** enforced; keep it local.

4. qector-doctor : 15-check environment diagnostic

Reports, per check, `PASS` / `WARN` / `FAIL`, plus an explanation. Answers *why* a decoder is unavailable instead of failing at decode time. The full set is self-documenting via `qector-doctor --json`. Representative checks:

1. Python version (3.9 to 3.13)
2. `qector_decoder_v3` importable, version == 1.0.0
3. wheel digests vs PyPI (optional `--check-hashes`)
4. NumPy bounds (`>=1.24.0`, `<2.3`)
5. licence resolution state (env/key/file), `key_status == "valid"` per tier
6. CPU core decoder availability (always PASS)
7. AVX2 batch path available
8. CUDA device probe (`cuda_is_available()`); WDDM vs TCC note
9. CUDA batch decode vs CPU batch equality on one synthetic shot
10. CuPy present / `gpu_available()`
11. OpenCL probe (`False` on wheels; documented)
12. stim import + DEM conversion works
13. sinter entry points resolvable
14. MCP server cold-start + `get_decoder_info` round-trip
15. write access to `QECTOR_DATA_DIR` (managed locations)

Exit `0` when all checks PASS; `1` on any FAIL/WARN; JSON report via `--json`.

PART IV: How to benchmark (methodology)

This part is a **methodology guide only**. It tells you how to measure decoding throughput and logical error rate on your own hardware with the tools the wheel ships. This manual publishes **no benchmark results**: every number you produce is system-specific and must be labelled as such (Part IV section 5).

1. Principles: honest, reproducible, system-specific

Before any measurement, agree with yourself (and your collaborators) on the following:

- **Throughput is not a property of the decoder alone.** It depends on the CPU model and microcode, RAM bandwidth, GPU model and driver stack (WDDM vs TCC), the OS scheduler, power management (turbo, DVFS), batch size, code family, distance and noise level. A number you measure on your laptop is a property of *that laptop at that moment*, not of the software.
- **Always fix the noise model.** Compare runs at the same physical error probability (e.g. `--noise 0.001`) and, for LER work, the same noise model. The `ler` module enforces this: `ler.assert_comparable(results)` returns the shared noise model or raises `NoiseModelMismatch` when rows mix models. Use it before any comparison.
- **Use deterministic seeds.** Every tool in this part accepts a `seed`. Keep it in the output. Seedless runs are not reproducible and not comparable.
- **Record the environment.** Machine model, CPU/GPU, driver version, OS, Python version, package version, seed, date, power mode. Without this, a result is uninterpretable.
- **Never subtract baselines you did not measure on the same box.** "X is N times faster than Y" is only meaningful when X and Y ran on the same hardware, same batch, same noise, same shot count, same process state.

2. Throughput via `qector bench` (CLI)

The CLI measures end-to-end decode throughput on a generated rotated surface-code circuit (Stim in-memory, no files required):

```
qector bench --distance 5 --rounds 5 --shots 10000 --decoder blossom --noise 0.001
```

Step-by-step:

1. Pick the code size (`--distance`) and rounds (`--rounds`) that match your target workload.
2. Set `--noise` to the physical error rate of your noise model. Keep it constant across the runs you will compare.
3. Choose `--shots` so the run lasts seconds, not milliseconds (for example 10,000 for a quick check, 100,000+ for a steadier figure). Short runs are dominated by construction and JIT/OS noise.
4. Repeat the same command 3 times and report the median, never the best run.
5. Record the machine state (turbo on/off, AC power, background load) in your notes.

The printed line (e.g. `shots/s` for the chosen family, distance and rounds) is a throughput estimate for that exact configuration. To compare families, change only `--decoder` and keep everything else identical.

3. LER via the Python `ler` module

Logical error rate is the accuracy metric: the fraction of shots whose decoded correction fails against the code's logical observables. Use the `ler` module directly from Python:

```
python

from qector_decoder_v3 import ler

# Code-capacity LER at one physical rate (i.i.d. bit-flip noise)
res = ler.estimate_ler(code, decoder, p=0.01, shots=100_000, seed=0)

# Circuit-level LER on a rotated surface code (gate/reset/measure noise)
res = ler.estimate_ler_circuit_level(
    distance=5, decoder="blossom", p=0.001, shots=100_000, rounds=5,
    seed=0, basis="x",
)

# Wilson 95% confidence interval for a binomial proportion
lo, hi = ler.wilson_ci(res.errors, res.shots)

# Enforce a single noise model before comparing any rows
shared = ler.assert_comparable([res1, res2]) # raises on mixture
```

Key facts:

- `estimate_ler` samples i.i.d. bit-flip errors, decodes in one warm batched decoder instance, and counts a logical failure whenever the residual `correction XOR error` carries logical content (code-observable matrix, E1/C6 criterion).
- Unfaithful corrections (those that do not reproduce the syndrome) are counted separately in `LerResult.unfaithful`, never silently folded into the LER. Zero unfaithful is the quality gate.
- `estimate_ler_circuit_level` is the measurement the QEC field benchmarks under and the one to use for **any cross-decoder comparison**: it samples detectors from a real Stim circuit with gate, reset and measurement noise over `rounds` rounds, and scores against the circuit's own logical observables.
- `run_threshold_sweep(code_factory, distances, p_values, decoder, shots, seed)` runs an LER-vs-p sweep per distance and reports the crossing where larger distance stops strictly lowering the LER (the threshold signature). `run_memory_experiment`, `run_competitive_suite` and `reference_validate` are available for full studies.
- `LerResult` fields: `decoder`, `code`, `physical_error_rate`, `shots`, `errors`, `unfaithful`, `seconds`, `seed`, `n_logical_qubits`, `noise_model`, `rounds`.

4. MCP benchmark procedure (`benchmark_decoder` , `run_ler_benchmark`)

The package MCP server (Part II section 7) exposes two benchmark tools. Start the server and send JSON-RPC 2.0 requests over stdio:

```
bash

python -c "import qector_decoder_v3; qector_decoder_v3.run_mcp_server()"
```

`benchmark_decoder` runs the Rust-native throughput suite for one decoder family on your check structure:

```

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "benchmark_decoder",
    "arguments": {
      "check_to_qubits": [[0, 1], [0, 2], [1, 3], [2, 3]],
      "n_qubits": 4,
      "n_samples": 10000,
      "seed": 42,
      "phys_error": 0.01,
      "decoder_type": "blossom"
    }
  }
}

```

Arguments:

Argument	Default	Meaning
check_to_qubits	required	check hyperedges of the code to benchmark
n_qubits	required	number of qubits
n_samples	10000	number of samples (bounded by server limit)
seed	42	determinism seed; keep it fixed for comparability
phys_error	0.01	physical error rate for syndrome generation
decoder_type	blossom	decoder family; strictly validated before any backend starts

The result is the structured suite report (samples, elapsed, per-shot statistics) for that decoder on that code. **License note:** this tool enforces an unlocked license (`enforce_unlocked`); without a valid key it returns a license error, not a result.

`run_ler_benchmark` runs the logical-error-rate benchmark across multiple code distances with no input arguments:

```

{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": { "name": "run_ler_benchmark", "arguments": {} }
}

```

It also requires an unlocked license. The result is the LER report across distances.

Procedure (do this in order):

1. Run `get_decoder_info` once to confirm server version and available families.
2. Run `get_backend_health` to know which tiers are actually available before benchmarking (e.g. CUDA tier only on a CUDA device; OpenCL always off in wheels).
3. Run `benchmark_decoder` with a fixed `seed` and `phys_error` for each family you compare; change only `decoder_type`.

4. Run `run_1er_benchmark` for the accuracy dimension.
5. Record everything from `get_server_env` (effective environment) plus your machine spec into the result log.
6. Clear the decoder cache (`clear_decoder_cache`) between families if you suspect cross-run state contamination.

5. Reporting checklist

When you publish or share any number produced with this package, attach:

- Package version (`qector-decoder-v3==1.0.0`), Python version, OS and platform.
- Machine: CPU model/microcode, RAM, GPU model and driver mode (WDDM/TCC).
- Power state (AC/battery, turbo, DVFS) and any other processes sharing the cores.
- Code: family, distance, rounds, check structure (or the code generator used).
- Noise: physical error probability and noise model; for LER, the result of `1er.assert_comparable` on all rows shown together.
- Seeds and shot counts; for throughput, the median of at least 3 runs.
- A sentence making explicit that figures are system-specific and not claimed as universal.

That is what "honest benchmarking" means with this package: same machine, same sampling, same noise, same shot count, same seeds, everything labelled.

Appendix A: Release history summary (0.5.0 to 1.0.0)

Ver-sion	Date	Highlights
1.0.0	2026-08-06	First stable v1. Stability tiers; CS-OSD + LLR damping; Relay-BP; <code>cluster_bposd</code> for colour codes; <code>CUDABatchDecoder(precision="f64")</code> ; <code>SparseBlossomDecoder</code> zero-allocation; <code>qector</code> CLI + <code>qector-doctor</code> ; entry points for Sinter/qiskit; <code>pymatching</code> submodule shim; licence-key API. 15 wheels (cp39 to cp313, 3 platforms), Sigstore.
0.7.1	2026-08-04	Patch: CLI <code>qector decode</code> crash fix (nonexistent import); MCP <code>ping</code> implemented; MCP no longer responds to notifications.
0.7.0	2026-08-01	<code>AmbiguityClusterDecoder</code> , <code>TwoStageDecoder</code> , <code>ColourCodeDecoder</code> ; Relay-BP; weighted UF on GPU + <code>edge_weights</code> ; <code>DemModel.make_decoder</code> (9 families); <code>qector</code> CLI; licence keys; belief matching faithful <code>from_numpy_h</code> ; exact log-domain BP; Rust core crash safety (6 panic paths removed).
0.6.9	2026-07-26	Faithful <code>from_numpy_h</code> for belief matching; exact log-domain BP default; OSD-1/2; licence hardening (malformed tokens <code>False</code>); v2 tokens with tier+expiry.
0.6.8	2026-07-22	Ed25519 <code>verify_license_token</code> ; auto policy dispatches qLDPC/bicycle to BP-OSD; <code>numpy>=1.24,<2.3</code> ; yanked 0.6.7 same day; 0.6.5 broke import on all wheels (fixed in 0.6.6); 0.6.3 to 0.6.5 yanked.
0.6.x early	2026-07	<code>DecoderPool</code> , <code>decode_mmap</code> , Workbench orchestrator surface, first GPU batch path.
0.5.x	2026	First PyPI line: Blossom/UF/BpOsd, stim/sinter bridges, narrow wheel matrix.

Appendix B: Wheel SHA-256 digest matrix (1.0.0, canonical)

Wheel	Size (B)	SHA-256 (full)
cp39 macosx_11_0_arm64	1,921,895	44284e2d57208637d499017ddfa940e3c61c9302b1ba06c322490aaa92e4f20a
cp39 manylinux_2_17_x86_64	2,169,453	3c9e847855a5aabcc721fcc056394264d0b63dfa71ac72902e1bd9ac35e47653
cp39 win_amd64	2,003,644	20efe31fa8629098f24f5a6306412b2c1f19ed00cb53c194440aa0162d33403e
cp310 macosx_11_0_arm64	1,920,360	08efc3944299c912d26ab76f7fe79684e859877981ff555c52b6ce89b6fa5f01
cp310 manylinux_2_17_x86_64	2,168,178	6a7f1a23ea20c1adb56754750e02590e94e32cac506567781a278850b216e30c
cp310 win_amd64	2,002,189	3d045a87954aea8f7ef83407bb90ddc32d6b42b44ed2d712420b6b18bdbec870
cp311 macosx_11_0_arm64	1,920,318	b3a6260b4daa12552ae7cc4a899e337361ec3b7a2dec71b291ab253252b322c2
cp311 manylinux_2_17_x86_64	2,167,962	9ee1da96e08e6e841e6d8768a830e253c229380e41454f26971d8bd4cc385575
cp311 win_amd64	2,001,836	7f198c7d9ca8f28c461f907d2ec4f41464198446abc2239c894b550bce7d4f98
cp312 macosx_11_0_arm64	1,908,684	64a9355daecb307affa098a8e17ad69b8c4486933bc09f3da8c46947b320979e
cp312 manylinux_2_17_x86_64	2,162,102	dca105a1cddbaf802cf8426e9f3a414ee1a3965380549531df88e3f06484276f
cp312 win_amd64	1,997,937	62c6569b656c584344168a4478e91912866a015542d06d6fb8d327deb6e377cd
cp313 macosx_11_0_arm64	1,908,439	db730d84c3006a3640e18b4fe4716161303198b1cebe2f596cf569e4e178dce4
cp313 manylinux_2_17_x86_64	2,162,223	02812bc01a2dff105b144896b1216e164bc67939483b6036c3ebcbcf6324f94a
cp313 win_amd64	1,998,089	a8f09b6849a006f8c1c540be4a44d5aab12f9ae579f74aabb98c1f4336bb6d88

Canonical machine-readable list. `pypi_v100_wheels.json` (in this package and on the PyPI release) carries every wheel's filename, exact size, full sha256 and the `files.pythonhosted.org` URL. Pin your deployment to the digest of your platform row from that JSON and verify with `sha256sum -c`.

Appendix C: License terms summary

- **PolyForm Noncommercial License 1.0.0.** You may use/modify/distribute (source form) for personal, academic, educational and non-commercial research; **you may not** use it commercially: company use, SaaS/hosted API, OEM redistribution/embedding, paid consulting, or funded-institutional work. That requires a commercial licence (`qector.store/pricing`).
- The verification package, this manual and linked report are licensed **CC-BY-4.0**.

Appendix D: Citation

Cite the package for code-specific results:

```
bibtex
@software{lessard2026qector,
  author = {Guillaume Lessard},
  title = {{QECTOR Decoder v3}: Rust/Python Quantum Error Correction Decoding Platform},
  year = {2026},
  version = {1.0.0},
  url = {https://www.qector.store},
  note = {Source-available under PolyForm Noncommercial 1.0.0.}
}
```

Cite the manual:

```
bibtex
@misc{lessard2026manual,
  author = {Guillaume Lessard},
  title = {{QECTOR Decoder v3} User Manual and Extended Reference, v1.0.0},
  year = {2026},
  howpublished = {https://qector.store / PyPI wheel manifest},
  note = {Companion to \cite{lessard2026qector}}
}
```

Text form: Lessard, G. (2026). *QECTOR Decoder v3, Official User Manual, v1.0.0*. <https://qector.store>. Distributed with `qector-decoder-v3==1.0.0`.

Appendix E: Licensing

QECTOR Decoder v3 is source-available under the PolyForm Noncommercial License 1.0.0 (see LICENSE). Personal, academic, educational, and non-commercial research use is allowed. Company use, funded institutional work, SaaS, hosted API deployment, OEM integration, redistribution, paid consulting, or commercial benchmarking requires a commercial license.

Pricing & tiers: <https://www.qector.store/pricing> Direct purchase: Buy via Stripe Contact: admin@qector.store

```
bibtex
@software{lessard2026qector,
  author = {Guillaume Lessard},
  title = {{QECTOR Decoder v3}: Rust/Python Quantum Error Correction Decoding Platform},
  year = {2026},
  version = {1.0.0},
  url = {https://www.qector.store},
  note = {Source-available under PolyForm Noncommercial 1.0.0. Commercial license required for commercial use.}
}
```